P3351R2

`views::scan`

Published Proposal, 2025-01-12

This version:
https://wg21.link/P3351R2

Author:
Yihe Li

Audience:
SG9

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Target:
C++26

Source:
github.com/Mick235711/wg21-papers/blob/main/P3351/P3351R2.bs

Issue Tracking:
GitHub Mick235711/wg21-papers

Table of Contents

8.1.1 SG9 Review, Wrocław (2024-11)

References

Normative References

Informative References

This paper proposes the `views::scan` range adaptor family, which takes a range and a function that takes the current element *and* the current state as parameters. Basically, `views::scan` is a lazy view version of `std::inclusive_scan`, or `views::transform` with a stateful function. To make common usage of this adaptor easier, this paper also proposes `views::partial_sum`, which is `views::scan` with the binary function defaulted to `+`.

The `views::scan` adaptor is classified as a Tier 1 item in the Ranges plan for C++26 ([P2760R1]).

§ 1. Revision History

§ 1.1. R2 (2025-01 pre-Hagenberg Mailing)

- Redesigned the entire paper, following SG9's feedback in Wrocław (2024-11). Significant changes since [P3351R1] are:
  - `views::scan` is now equivalent to `std::inclusive_scan`, both in terms of argument sequence (`rng, func, init`) and also the semantics.
  - `views::prescan` is removed as it did not gain enough support and have no prior experience in the STL.
  - The prior arts section is redesigned:
    - Added STL algorithm to prior arts section, and also reformat the table.
    - More languages like Kotlin and Haskell are added.
  - After the parameter sequence change, conflict between range and initial value is no longer an issue, thus removing this workaround which previously prevent the with-initial-value version to overload with the normal version. As a result, now `scan` and `partial_sum` supports an optional initial value.
  - A section is added to explore the relationship with parallel range algorithms in general.
  - Group the wording question into different groups based on audience.
- Added poll results section and several review results sections.
- Added some minor correction and discussion on reference type.

- Added some discussion on naming comflicts with [P1729R5].
- Several wording fixes:
  - Fixed the mismatch template arguments for `scan_view` in synopsis.
  - Add *italics* for exposition-only concepts.
- Preemptively add `reserve_hint` members in anticipation of [P2846R5]'s acceptance.
- Rebase onto latest draft [N5001].

1.2. R1 (2024-10 pre-Wrocław Mailing)

- Several wording fixes:
  - Added the missing `noexcept` to `end()`.
  - Refactored the constraints of `scan_view` out into its own concept, such that it tests for both assignability from `range_reference_t<R>` and the invoke result of `f`.
  - Replace `regular_invocable` with `invocable`.
  - Pass by move in `scan_view`'s constructor and when invoking the function.
- Rebase onto latest draft [N4988].

1.3. R0 (2024-07 post-St. Louis Mailing)

- Initial revision.

2. Motivation

The motivation for this view is given in [P2760R1] and quoted below for convenience:

> If you want to take a range of elements and get a new range that is applying `f` to every element, that's `transform(f)`. But there are many cases where you need a `transform` to that is stateful. That is, rather than have the input to `f` be the current element (and require that `f` be `regular_invocable`), have the input to `f` be both the current element and the current state.
>
> For instance, given the range `[1, 2, 3, 4, 5]`, if you want to produce the range `[1, 3, 6, 10, 15]` - you can't get there with `transform`. Instead, you need to use `scan` using `+` as the binary operator. The special case of `scan` over `+` is partial_sum.
>
> One consideration here is how to process the first element. You might want `[1, 3, 6, 10, 15]` and you might want `[0, 1, 3, 6, 10, 15]` (with one extra element), the latter could be called a `prescan`.

This adaptor is also present in ranges-v3, where it is called `views::partial_sum` with the function parameter defaulted to `std::plus{}`. However, as [P2760R1] rightfully pointed out, `partial_sum` is probably not suitable for a generic operation like this that can do much more than just calculating partial sum, similar to how `accumulate` is not a suitable name for a general fold. Therefore, the more generic `scan` name is chosen to reflect the nature of this operation. (More discussion on the naming are present in the later sections.)

3. Design

3.1. What Is Being Proposed?

```
std::vector vec{1, 2, 3, 4, 5};
std::println("{}", vec | views::scan(std::plus{})); // [1, 3, 6, 10, 15]
std::println("{}", vec | views::partial_sum); // [1, 3, 6, 10, 15]
std::println("{}", vec | views::scan(std::plus{}, 10)); // [11, 13, 16, 20,
std::println("{}", vec | views::partial_sum(10)); // [11, 13, 16, 20, 25]
```

3.2. Background

Scan is a pretty common operation in functional programming. On its simplest form, scanning a sequence `x0, x1, x2, ...` gives you back the sequence `x0, x0 + x1, x0 + x1 + x2, ...`, essentially doing a prefix sum. When generalizing `+` to an arbitrary binary function, scan become a

form of "partial" fold, in which fold-like operation is performed but with intermediate result preserved.

There are two forms of scan defined in a mathematical sense: *inclusive* scan and *exclusive* scan. An inclusive scan includes `i`-th term of the original sequence when computing the `i`-th term of the resulting sequence, while the exclusive scan does not. Inclusive scan may optionally include an initial seed `s` that will be used to compute the first output term, while exclusive scan requires an initial seed as the first output term directly. When summarized in a table, both forms of scan looks like this:

Original	`x0`	`x1`	`x2`	`x3`
Inclusive	`x0`	`f(x0, x1)`	`f(f(x0, x1), x2)`	`f(f(f(x0, x1), x2), x3)`
Inclusive With Seed	`f(s, x0)`	`f(f(s, x0), x1)`	`f(f(f(s, x0), x1), x2)`	`f(f(f(f(s, x0), x1), x2), x3)`
Exclusive	`s`	`f(s, x0)`	`f(f(s, x0), x1)`	`f(f(f(s, x0), x1), x2)`

We can see clearly that both forms of scan are easily interconvertible, as shifting the inclusive scan result right once, prepend the initial seed, and drop the last term gives you the exclusive scan result. Another critical characteristic of exclusive scan is that they disregard the last input term to preserve the length of the sequence.

In light of this, a third, more general scan semantics was implemented by some languages (like Haskell's `scanl`): Prepend the initial seed, but does not drop the last term. This is called `prescan` semantics in previous revisions of this proposal, and has the notable characteristic that the resulting sequence will be one longer than the input. When applied to the sequence `[1, 2, 3, 4, 5]` and using addition as the binary function, the three scan semantics gives the following results:

Original	`[1, 2, 3, 4, 5]`
Inclusive	`[1, 3, 6, 10, 15]`
Inclusive With Seed `10`	`[11, 13, 16, 20, 25]`
Exclusive With Seed `10`	`[10, 11, 13, 16, 20]`
`prescan` With Seed `10`	`[10, 11, 13, 16, 20, 25]`

C++ STL provides `std::partial_sum` and `std::inclusive_scan` (latter since C++17) with the inclusive semantics, and `std::exclusive_scan` (since C++17) with the exclusive semantics.

Previous revisions of this proposal proposes both the inclusive and the `prescan` semantics, while the current revision only proposes the inclusive semantics. As can be seen in the Prior Arts sections below, inclusive semantics is the most commonly used and provided semantic in most languages, and other two semantics can be easily constructed by manipulating the inclusive result with existing range adaptors:

```
exclusive_scan(rng, func, init) = concat(single(init), scan(rng, func, init
prescan(rng, func, init) = concat(single(init), scan(rng, func, init))
```

Therefore, the author thinks that providing only the inclusive semantics is sufficient.

§ 3.3. Prior Art

The scan adaptor/algorithm had made an appearance in many different libraries and languages:

- range-v3 has a `views::partial_sum` adaptor that don't take initial seeds, but takes arbitrary function parameter (defaults to `+`).
    - range-v3 also has an `views::exclusive_scan` adaptor that implements exclusive scan semantics. Its initial seed parameter is mandatory, but also takes an optional function parameter.
- STL has `std::partial_sum` algorithm in `<numeric>` since C++98, and `std::[in, ex]clusive_scan` algorithms in `<numeric>` since C++17.
- `tl::ranges` also has an implementation of `views::partial_sum`.
- Python has an `itertools.accumulate` algorithm that optionally takes initial seeds (with a named argument), and takes arbitrary function parameter (defaults to `+`).
    - Furthermore, NumPy provides `cumsum` algorithm that returns a partial sum (without function parameter), and an `ufunc.accumulate` algorithm that performs arbitrary scans with arbitrary function parameter that have no default. Neither of those algorithms takes an initial seed.
- Rust has an `iter.scan` adaptor that takes initial seeds and arbitrary function parameters (no defaults provided). Kotlin also provides a `scan` member that works similarly.
- Haskell provides two variants of left scan: `scanl1` and `scanl`. The former performs an inclusive scan, but does not permit an initial seed. The latter performs a `prescan`-like operation

that preserve both the initial seed provided and the last fold result, resulting in a one-larger range than passed in.

Summarized in a table (without additional note, all functions provide inclusive scan semantics):

Library	Signature	Parameter	Function	With Default	Initial Seed
Lazy Adaptors					
range-v3 / `tl::ranges`	`partial_sum`	`(rng[, fun])`	✅	✅	❌
	`exclusive_scan` (Exclusive semantics)	`(rng, init[, fun])`	✅	✅	✅
Proposed	`scan`	`(rng, func[, init])`	✅	❌	✅
	`partial_sum`	`(rng[, init])`	❌	N/A	✅
Eager Algorithms					
STL	`std::partial_sum`	`(rng, out[, func])`	✅	✅	❌
	`std::inclusive_scan`	`(rng, out[, func[, init]])`	✅	✅	✅
	`std::exclusive_scan` (Exclusive semantics)	`(rng, out, init[, func])`	✅	✅	✅
Python `itertools`	`accumulate`	`(rng[, fun[, init=init]])`	✅	✅	✅
NumPy	`cumsum`	`(rng)`	❌	N/A	❌
	`<func>.accumulate`	`(rng)`	✅	❌	❌
Rust	`iter.scan`	`(init, func)`	✅	❌	✅
Kotlin	`<rng>.scan`	`(init, func)`	✅	❌	✅
Haskell	`scanl1`	`(func, rng)`	✅	❌	❌
	`scanl` (`prescan` semantics)	`(func, init, rng)`	✅	❌	✅

§ 3.4. Alternative Names

The author thinks `views::scan` and `views::partial_sum` are pretty good names. The former have prior example in `std::[in,ex]clusive_scan` and in Rust/Kotlin, and is a generic enough name that will not cause confusion. The latter also have prior example in `std::partial_sum`, and partial sum is definitely one of the canonical terms of describing this operation.

However, one possible naming conflict recently has arisen: `std::scan` ([P1729R5]). Here, the name `scan` refers to a completely different concept: scanning user's input, and `std::scan` is proposed as an upgraded version of the `scanf` family and uses `std::format`-like syntax to parse variables from a given string input:

```
auto result = std::scan<std::string, int>("answer = 42", "{} = {}");
const auto& [key, value] = result->values();
// key == "answer", value == 42
```

As noted above, this interpretation of the name `scan` also has precedence in the standard library: the `scanf` family, so this meaning also makes some sense.

Notice that `std::scan` and `views::scan` do not conflict with each other since these two names are in different namespaces. However, it may be of interest to SG9/LEWG to rename `views::scan` to a different name in order to avoid user confusion about two unrelated facilities being named the same. In light of this, the following list of alternative names is provided for consideration:

Alternative names considered for `views::scan`: (in order of decreasing preference)

- `views::partial_fold` (suggested by [P2214R2], and makes the connection with `ranges::fold` clear): Pretty good name, since `scan` is just a `fold` with intermediate state saved. However, the correct analogy is actually `ranges::fold_left_first`, and `ranges::fold_left` actually corresponds to an exclusive scan, which the author felt may cause some confusion.
- `views::inclusive_scan`: Makes the equivalence with the STL numeric algorithm clear, and have no conflict potential with `scanf` anymore. However, the author felt that this name is a bit too long.
- `views::accumulate`, `views::fold`, `views::fold_left`: The output is a range instead of a number, so using the same name probably is not accurate. The latter two also suffer from the same displaced correspondence problem.

Alternative names considered for `views::partial_sum`: (in order of decreasing preference)

- `views::prefix_sum`: Another canonical term for this operation, but doesn't have prior examples.
- `views::cumsum` or `views::cumulative_sum`: Have prior example in NumPy, but in the spirit of Ranges naming we probably need to choose the latter which is a bit long.

Overall, the author doesn't find any of these options particularly intriguing and opts to propose the original naming of `scan` and `partial_sum`. This does not actually conflict with `std::scan`, and

besides, the standard library is already adopting two meanings of `scan` by introducing both `scanf` and `std::[in, ex]clusive_scan`, so it shouldn't be that much of a confusion risk to the users.

3.5. Left or Right Fold?

Theoretically, there are two possible direction of scanning a range:

```
// rng = [x1, x2, x3, ...]
scan_left(rng, f, i) // [x1, f(x1, x2), f(f(x1, x2), x3), ...]
scan_right(rng, f, i) // [x1, f(x2, x1), f(x3, f(x2, x1)), ...]
```

Both are certainly viable, which begs the question: Should we provide both?

On the one hand, `ranges::fold` provided both the left and the right fold version, despite the fact that right fold can be simulated by reversing the range and the function parameter order. However, here, the simulation is even easier: just reversing the order of the function parameters will turn a left scan to a right scan.

Furthermore, all of the mentioned prior arts perform left scan, and it is hard to come up with a valid use case of right scan that cannot be easily covered by left scan. Therefore, the author only proposes left scan in this proposal.

3.5.1. SG9 Review, Wrocław (2024-11)

SG9 agreed that only left scan is needed, since a right scan can be easily simulated by flipping the arguments of the function provided. The fundamental difference between `fold` and `scan` here is that `fold` requires both reversing the range *and* flipping the arguments when changing from left to right, while `scan` only requires the latter.

3.6. More Convenience Aliases

Obviously, there are more aliases that can be provided besides `partial_sum`. The three most useful aliases are:

```
std::vector<int> vec{3, 4, 6, 2, 1, 9, 0, 7, 5, 8}
partial_sum(vec) // [3, 7, 13, 15, 16, 25, 25, 32, 37, 45]
partial_product(vec) // [3, 12, 72, 144, 144, 1296, 0, 0, 0, 0]
```

```
running_min(vec) // [3, 3, 3, 2, 1, 1, 0, 0, 0, 0]
running_max(vec) // [3, 4, 6, 6, 6, 9, 9, 9, 9, 9]
```

Which are the results of applying `std::plus{}`, `std::multiplies{}`, `ranges::min`, and `ranges::max`.

Looking at the results, these are certainly very useful aliases, even as useful as `partial_sum`. However, as stated above, all of these three aliases can be achieved by simply passing the corresponding function object (currently, `min`/`max` doesn't have a function object, but `ranges::min` and `ranges::max` become useable after [P3136R1] was adopted in Wrocław (2024-11)) as the function parameter, so I'm not sure it is worth the hassle of specification. Therefore, currently this proposal do not include the other three aliases, but the author is happy to add them should SG9/LEWG request.

As for `partial_sum` itself, there are several reasons why this alias should be added, that is certainly stronger than the case for `product`, `min` and `max`:

- All existing implementation of scan-like algorithm defaults the function argument to `+` whenever there is a default.
- `std::partial_sum` exists
- Partial sum is one of the most studied and used concept in programming, arguably even more useful than running min/max.

One remaining question is whether `partial_sum` should be a standalone adaptor alias, or should it be folded into `scan` as the default function parameter. On the one hand, range-v3's `views::partial_sum` does this folding by using a function parameter defaults to `std::plus{}`. On the other hand, the author thinks that this will definitely cause some confusion to the users, due to the fact that a generic name like `scan` defaults to `std::plus` as its function parameter:

```
vec | views::scan // what should this mean?
```

This is similar to the scenario encountered by `ranges::fold` ([P2322R6]), where despite the old algorithm `std::accumulate` took `std::plus` as the default function parameter, `ranges::fold` still choose to not have a default. The author feels that the same should be done for `views::scan` (i.e. no default function), and introduce a separate `views::partial_sum` alias that more clearly convey the intent.

§ 3.7. Range Properties

All three proposed views are range adaptors, i.e. can be piped into. `partial_sum` is just an alias for `scan(std::plus{})`, so it will not be mentioned in the below analysis.

§ 3.7.1. Reference and Value Type

Consider the following:

```
std::vector<double> vec{1.0, 1.5, 2.0};
vec | views::scan(std::plus{}, 1);
```

Obviously, we expect the result to be `[1.0, 2.0, 3.5, 5.5]`, not `[1, 2, 3, 5]`, therefore for `scan` we cannot just use the initial seed's type as the resulting range's reference/value type.

There are two choices we can make: (for input range type `Rng`, function type `F` and initial seed type `T`)

1. Just use the reference/value type of the input range. This is consistent with `std::partial_sum`. (range-v3 also chose this approach, using the input range's value type as the reference type of `scan_view`)
2. Be a bit clever, and use `decay_t<invoke_result_t<F&, T, ranges::range_reference_t<Rng>>>`. In other words, the return type of `func(init, *rng.begin())`.

Note that the second option don't really covers all kind of functions, since it is entirely plausible that `func` will change its return type in every invocation. However, this should be enough for nearly all normal cases, and is the decision chosen by [P2322R6] for `ranges::fold_left[_first]`. For `scan` without an initial seed, this approach can also work, by returning the return type of `func(*rng.begin(), *rng.begin())`.

Although the second choice is a bit complex in design, it also avoids the following footgun:

```
std::vector<int> vec{1, 4, 2147483647, 3};
vec | views::scan(std::plus{}, 0L);
```

With the first choice, the resulting range's value type will be `int`, which would result in UB due to overflow. With the second choice the resulting value type will be `long` which is fine. (Unfortunately, if you omit the initial seed here it will still be UB, and that cannot be fixed.)

The second choice also enables the following use case:

```
// Assumes that std::to_string also has an overload for std::string that ju
std::vector<int> vec{1, 2, 3};
vec | views::scan(
    [](const auto& a, const auto& b) { return std::to_string(a) + std::to_s
    "2"
) // ["21", "212", "2123"]
```

Given that `ranges::fold_left[_first]` chose the second approach, the author thinks that the second approach is the correct one to pursue. In other words, the value type of `scan_view` will be `decay_t<invoke_result_t<F&, T, ranges::range_reference_t<Rng>>>`, and the reference type will be `const value_type&`.

3.7.2. Category

At most forward. (In other words, forward if the input range is forward, otherwise input.)

The resulting range cannot be bidirectional, since the function parameter cannot be applied in reverse. A future extension may enable a `scan` with both forward and backward function parameter, but that is outside the scope of this paper.

3.7.3. Common

Never.

This is consistent with range-v3's implementation. The reasoning is that each iterator needs to store both the current position and the current partial sum (generalized), so that the `end()` position's iterator is not readily available in O(1) time.

3.7.4. Sized

If and only if the input range is sized. The reported size is always equal to the input range's size.

Also, given that [P2846R5] is on track to be adopted for C++26, `scan_view` also adapted that proposal by adding `reserve_hint()` members who forwards the underlying view's `reserve_hint()` result appropriately.

3.7.5. Const-Iterable

Similar to `views::transform`, if and only if the input range is const-iterable and `func` is `const`-invocable.

3.7.6. Borrowed

Never.

At least for now. Currently, `views::transform` is never borrowed, but after [P3117R1] determined suitable criteria for storing the function parameter in the iterator, it can be conditionally borrowed.

Theoretically, the same can be applied to `scan_view` to make it conditionally borrowed by storing the function and the initial value inside the iterator. However, the author would like to wait until [P3117R1] lands to make this change.

3.8. Feature Test Macro

This proposal added a new feature test macro `__cpp_lib_ranges_scan`, which signals the availability of both `scan` and `partial_sum` adaptors.

3.9. Freestanding

[P1642R11] included nearly everything in `<ranges>` into freestanding, except `views::istream` and the corresponding view types. However, range adaptors added after [P1642R11], like `views::concat` and `views::enumerate` did not include themselves in freestanding.

The author assumes that this is an oversight, since I cannot see any reason for those views to not be in freestanding. As a result, the range adaptor proposed by this paper will be included in freestanding. (Adding freestanding markers to the two views mentioned above is probably out of scope for this paper, but if LWG decides that this paper should also solve that oversight, the author is happy to comply.)

3.10. Future Extensions

Several future extensions to the proposed `scan` adaptor are possible and seen in other languages. These are outside the scope of this proposal and are only listed here as a record for inspiration.

One obvious extension is to provide a variant that allows functions that operate on indexes, similar to Kotlin's `scanIndexed` member:

```
>>> scanIndexed(["a", "b", "c"], "start", lambda index, acc, elem: acc + f'
["start, 0: a", "start, 0: a, 1: b", "start, 0: a, 1: b, 2: c"]
```

This is already achievable in C++ by using `views::enumerate` and manually unpacking the tuple elements inside the function. A direct `scan_indexed` adaptor would be more convenient but ultimately does not provide new functionalities that cannot be achieved using existing tools, so it is not proposed.

Another extension concerns the parallelism potential for the scan algorithm. At first glance, scan seems to be a completely serial algorithm that cannot be paralleled, as each term depends on the previous intermediate result. However, by iteratively performing the scanned function, the algorithm can actually be paralleled and can be completed in logarithmic time regarding to the number of elements when parallel resources are abundant. See the Wikipedia page for details.

In light of this, concerns are raised about the proposed `scan` adaptor in that it does not permit this kind of reorder for paralleled execution:

```
// Previous: two passes, two launches, each paralleled
ranges::inclusive_scan(par, rng, rng);
ranges::transform(par, rng, some_func);

// After: one pass but actually serial
ranges::transform(par, rng | views::partial_sum, some_func);
```

This is due to the fact that `scan_view` is at most forward, and thus, all the scanning operations can only be performed in sequence.

However, the author is unable to find a satisfactory solution to this problem. With the C++ iterator model, efficient paralleled algorithm execution is **only** possible with random access iterator/range inputs, as referring to an arbitrary point in the iteration space at a constant time is often an integral part of the algorithm. Although C++17 parallel algorithms generally only require forward iterators, in practice, most implementations (like libstdc++ and libc++) will *de facto* require random access iterators to enable parallelism and silently fall back to serial execution with lower categories. Existing third-party parallel libraries like OpenMP and oneTBB also nearly always require random access iterators in their algorithms.

Yet, it is impossible to make `scan_view` a random access range. Doing so will not only require a way to get the previous results (probably through an additional reversal functor) but also requires efficient computation of the N-th element in the output range, which by definition requires N execution of the function and thus is impossible to be made efficient. The parallel algorithm for scan requires a substantially different iterator/adaptor model that allows efficient and arbitrary chunking of inner operations, which the C++ iterator model does not permit.

This is a similar problem faced by `views::filter`. On its own, `filter_view` is at most bidirectional regardless of inputs, which makes it impossible to be execute efficiently using standard parallel algorithms and most third-party libraries as it is never random access. However, a natural parallel algorithm exists for `filter_view`: just filter whatever element you are processing in parallel and exit the current thread/worker if the filter returns `false`. Unfortunately, this is not expressible as the iterator advancement is strictly tied to the filter operation.

There are several possible mitigation strategies for this problem, but none of them are complete:

- Add named algorithms. For instance, given that `filter_view` is not paralleled, C++17 provided a parallel `copy_if` to provide the desired semantics. However, algorithms are not composable (which is the whole reason that range adaptors are introduced), and using parallel STL algorithms like `copy_if` and `inclusive_scan` in lieu of adaptors leads to suboptimal performance (due to `copy_if` needs some synchronization for writing the output while `filter` does not) and the multi-launch problem described above.
- Add named algorithms for compositions. For instance, to solve the transform + scan multi-launch problem, STL directly provides `transform_{in,ex}clusive_scan`, whose paralleled version can perform two operations in one launch. However, compositions are infinite, and STL cannot provide every composition as named algorithms (the above scan + transform operation, for instance, is not provided).
- Special-case the range adaptors. For instance, paralleled `ranges::transform` can identify the `scan_view` being passed in and utilize `base()` and functor members to do the parallelization. However, this solution is brittle, cannot handle complex pipelines, and requires substantial work for all algorithms.

Ultimately, the C++20 range adaptor is not a good abstraction for parallelism, and complex pipelines using adaptors are inherently only able to be executed serially. The parallelism problem is not unique to `scan_view`, and substantial changes to both the adaptor and iterator model (perhaps somehow separate the advancement and compute stages) are required to enable sufficient parallelism opportunities even for long pipeline compositions, which are best explored in a separate proposal. Fortunately, at least the current range parallel algorithm proposal [P3179R4] wisely restricted the

input to be random access, so the above scan + transform invocation will result in a compile error instead of silently degrading to serial performance.

§ 4. Implementation Experience

The author implemented this proposal in Compiler Explorer. No significant obstacles are observed.

§ 5. Questions To Resolve

§ 5.1. Questions for SG9

- Should the naming be `views::inclusive_scan` to avoid conflict?
- Should the function parameter have a default?
- Do we need `operator-` for `sized_sentinel_for`? Between two iterators of `scan_view` and/or between iterators of `scan_view` and iterators of base range? Between iterator and sentinel?
  - range-v3's `partial_sum_view` does not have `operator-`, but `tl::ranges`'s does.

§ 5.2. Questions for LEWG

- Currently, the current sum is cached in the iterator object (as implemented in range-v3). An alternative is to cache the current sum in the view object (as implemented in `tl::ranges`), such that each iterator only needs to hold a pointer to the parent view and an iterator to the current position. Which one should be chosen?
- `regular_invocable` or `invocable`? Currently the wording uses the latter (following range-v3), but `transform_view` used the former.

§ 5.3. Questions for LWG

- Due to never being a common range, `scan_view` simply have a single `end() const` member that returns `default_sentinel`. Is that the correct way to do this, or should I just use the end iterator of the base range?
- The extra `bool IsInit` parameter seems a bit clunky.

§ 6. Wording

The wording below is based on [N5001] with [P2846R5] already applied on top.

Wording notes for LWG and editor:

- Currently, the three views' definition reside just after `views::transform`'s definition and synopsis, due to the author's perception that they are pretty similar.
- The exposition-only concept *`scannable`* and *`scannable-impl`* is basically the same as *`indirectly-binary-left-foldable`* and *`indirectly-binary-left-foldable-impl`*; the only difference is that `F` is required to be move constructible instead of copy constructible.

§ 6.1. 17.3.2 Header `<version>` synopsis [version.syn]

In this clause's synopsis, insert a new macro definition in a place that respects the current alphabetical order of the synopsis, and substituting `20XXYYL` by the date of adoption.

```
#define __cpp_lib_ranges_scan 20XXYYL // freestanding, also in <ranges>
```

§ 6.2. 25.2 Header `<ranges>` synopsis [ranges.syn]

Modify the synopsis as follows:

```
// [...]
namespace std::ranges {
  // [...]
  // [range.transform], transform view
  template<input_range V, move_constructible F, bool IsInit>
    requires view<V> && is_object_v<F> &&
             regular_invocable<F&, range_reference_t<V>> &&
             can-reference<invoke_result_t<F&, range_reference_t<V>>>
  class transform_view; // freestanding

  namespace views { inline constexpr unspecified transform = unspecified; }

  // [range.scan], scan view
  template<input_range V, move_constructible F, move_constructible T, bool
    requires see below
```

```
  class scan_view; // freestanding

  namespace views {
    inline constexpr unspecified scan = unspecified; // freestanding
    inline constexpr unspecified prefix_sum = unspecified; // freestanding
  }

  // [range.take], take view
  template<view> class take_view; // freestanding

  template<class T>
    constexpr bool enable_borrowed_range<take_view<T>> =
      enable_borrowed_range<T>; // freestanding

  namespace views { inline constexpr unspecified take = unspecified; } // 
  // [...]
}
```

Editor's Note: Add the following subclause to 25.7 Range adaptors [range.adaptors], after 25.7.9 Transform view [range.transform]

§ 6.3. 25.7.[illegible] Scan view [range.scan]

§ 6.4. 25.7.[illegible].1 Overview [range.scan.overview]

1. `scan_view` presents a view that accumulates the results of applying a transformation function to the current state and each element.
2. The name `views::scan` denotes a range adaptor object ([range.adaptor.object]). Given subexpressions `E`, `F` and `G`, the expressions `views::scan(E, F)` and `views::scan(E, F, G)` are expression-equivalent to `scan_view(E, F)` and `scan_view(E, F, G)`, respectively.

[*Example 1*:

```
vector<int> vec{1, 2, 3, 4, 5};
for (auto&& i : std::views::scan(vec, std::plus{})) {
  std::print("{} ", i); // prints 1 3 6 10 15
}
```

```
for (auto&& i : std::views::scan(vec, std::plus{}, 10)) {
  std::print("{} ", i); // prints 11 13 16 20 25
}
```

-- *end example*]

3. The name `views::partial_sum` denotes a range adaptor object ([range.adaptor.object]). Given subexpression `E`, the expressions `views::partial_sum(E)` and `views::partial_sum(E, F)` are expression-equivalent to `scan_view(E, std::plus{})` and `scan_view(E, std::plus{}, F)`, respectively.

§ 6.5. 25.7.﷿.2 Class template `scan_view` [range.scan.view]

```
namespace std::ranges {
  template<typename V, typename F, typename T, typename U>
    concept scannable-impl =  // exposition only
      movable<U> &&
      convertible_to<T, U> && invocable<F&, U, range_reference_t<V>> &&
      assignable_from<U&, invoke_result_t<F&, U, range_reference_t<V>>>;

  template<typename V, typename F, typename T>
    concept scannable =  // exposition only
      invocable<F&, T, range_reference_t<V>> &&
      convertible_to<invoke_result_t<F&, T, range_reference_t<V>>,
             decay_t<invoke_result_t<F&, T, range_reference_t<V>>>> &&
      scannable-impl<V, F, T,
             decay_t<invoke_result_t<F&, T, range_reference_t<V>>>>;

  template<input_range V, move_constructible F, move_constructible T, bool
    requires view<V> && is_object_v<F> && is_object_v<T> && scannable<V, F,
  class scan_view : public view_interface<scan_view<V, F, T, IsInit>> {
  private:
    // [range.scan.iterator], class template scan_view::iterator
    template<bool> struct iterator; // exposition only

    V base_ = V(); // exposition only
    movable-box<F> fun_; // exposition only
    movable-box<T> init_; // exposition only

  public:
    scan_view() requires default_initializable<V> && default_initializable<
```

```
    constexpr explicit scan_view(V base, F fun) requires (!IsInit);
    constexpr explicit scan_view(V base, F fun, T init) requires IsInit;

    constexpr V base() const & requires copy_constructible<V> { return base
    constexpr V base() && { return std::move(base_); }

    constexpr iterator<false> begin();
    constexpr iterator<true> begin() const
      requires range<const V> && scannable<const V, const F, T>;

    constexpr default_sentinel_t end() const noexcept { return default_sent

    constexpr auto size() requires sized_range<V> { return ranges::size(bas
    constexpr auto size() const requires sized_range<const V>
    { return ranges::size(base_); }

    constexpr auto reserve_hint() requires approximately_sized_range<V>
    { return ranges::reserve_hint(base_); }

    constexpr auto reserve_hint() const requires approximately_sized_range<
    { return ranges::reserve_hint(base_); }
  };

  template<class R, class F>
    scan_view(R&&, F) -> scan_view<views::all_t<R>, F, range_value_t<R>, fa
  template<class R, class F, class T>
    scan_view(R&&, F, T) -> scan_view<views::all_t<R>, F, T, true>;
}
```

```
constexpr explicit scan_view(V base, F fun) requires (!IsInit);
```

1. *Effects*: Initializes `base_` with `std::move(base)` and `fun_` with `std::move(fun)`.

```
constexpr explicit scan_view(V base, F fun, T init) requires IsInit;
```

2. *Effects*: Initializes `base_` with `std::move(base)`, `fun_` with `std::move(fun)`, and `init_` with `std::move(init)`.

```
constexpr iterator<false> begin();
```

3. *Effects*: Equivalent to: `return iterator<false>{*this, ranges::begin(base_)};`

```
constexpr iterator<true> begin() const
  requires range<const V> && scannable<const V, const F, T>;
```

4. *Effects*: Equivalent to: `return iterator<true>{*this, ranges::begin(base_)};`

§ 6.6. 25.7.﷿.3 Class template `scan_view::iterator` [range.scan.iterator]

```
namespace std::ranges {
  template<input_range V, move_constructible F, move_constructible T, bool
    requires view<V> && is_object_v<F> && is_object_v<T> && scannable<V, F,
  template<bool Const>
  class scan_view<V, F, T, IsInit>::iterator {
  private:
    using Parent = maybe-const<Const, scan_view>; // exposition only
    using Base = maybe-const<Const, V>; // exposition only
    using ResultType = decay_t<invoke_result_t<maybe-const<Const, F>&, T, r

    iterator_t<Base> current_ = iterator_t<Base>(); // exposition only
    Parent* parent_ = nullptr; // exposition only
    movable-box<ResultType> sum_; // exposition only

  public:
    using iterator_concept =
      conditional_t<forward_range<Base>, forward_iterator_tag, input_iterat
    using iterator_category = see below; // present only if Base models for
    using value_type = ResultType;
    using difference_type = range_difference_t<Base>;

    iterator() requires default_initializable<iterator_t<Base>> = default;
    constexpr iterator(Parent& parent, iterator_t<Base> current);
    constexpr iterator(iterator<!Const> i)
      requires Const && convertible_to<iterator_t<V>, iterator_t<Base>>;

    constexpr const iterator_t<Base>& base() const & noexcept { return curr
    constexpr iterator_t<Base> base() && { return std::move(current_); }

    constexpr const value_type& operator*() const { return *sum_; }
```

```
    constexpr iterator& operator++();
    constexpr void operator++(int);
    constexpr iterator operator++(int) requires forward_range<Base>;

    friend constexpr bool operator==(const iterator& x, const iterator& y)
      requires equality_comparable<iterator_t<Base>>;
    friend constexpr bool operator==(const iterator& x, default_sentinel_t);
  };
}
```

1. If `Base` does not model `forward_range` there is no member `iterator_category`. Otherwise, the *typedef-name* `iterator_category` denotes:

- `forward_iterator_tag` if `iterator_traits<iterator_t<Base>>::iterator_category` models `derived_from<forward_iterator_tag>` and `is_reference_v<invoke_result_t<maybe-const<Const, F>&, maybe-const<Const, T>&, range_reference_t<Base>>>` is `true`;
- otherwise, `input_iterator_tag`.

```
constexpr iterator(Parent& parent, iterator_t<Base> current);
```

2. *Effects*: Initializes `current_` with `std::move(current)` and `parent_` with `addressof(parent)`. Then, equivalent to:

```
if (current_ == ranges::end(parent_->base_)) return;
if constexpr (IsInit) {
  sum_ = invoke(*parent_->fun_, *parent_->init_, *current_);
} else {
  sum_ = *current_;
}
```

```
constexpr iterator(iterator<!Const> i)
  requires Const && convertible_to<iterator_t<V>, iterator_t<Base>>;
```

3. *Effects*: Initializes `current_` with `std::move(i.current_)`, `parent_` with `i.parent_`, and `sum_` with `std::move(i.sum_)`.

```
constexpr iterator& operator++();
```

4. *Effects*: Equivalent to:

```
if (++current_ != ranges::end(parent_->base_)) {
  sum_ = invoke(*parent_->fun_, std::move(*sum_), *current_);
}
return *this;
```

```
constexpr void operator++(int);
```

5. *Effects*: Equivalent to `++*this`.

```
constexpr iterator operator++(int) requires forward_range<Base>;
```

6. *Effects*: Equivalent to:

```
auto tmp = *this;
++*this;
return tmp;
```

```
friend constexpr bool operator==(const iterator& x, const iterator& y)
  requires equality_comparable<iterator_t<Base>>;
```

7. *Effects*: Equivalent to: `return x.current_ == y.current_;`

```
friend constexpr bool operator==(const iterator& x, default_sentinel_t);
```

8. *Effects*: Equivalent to: `return x.current_ == ranges::end(x.parent_->base_);`

7. Poll Results

7.1. SG9 Review, Wrocław (2024-11)

Poll 1: We want a view with a convenient spelling for the semantics of `std::inclusive_scan` without initial value.

Strongly Favor	Favor	Neutral	Against	Strongly Against
1	5	3	0	0

- **Attendance**: 10
- **Author's Position**: Favor
- **Outcome**: Consensus.

Poll 2: We want a view with a convenient spelling for the semantics of `std::inclusive_scan` with initial value.

Strongly Favor	Favor	Neutral	Against	Strongly Against
1	7	1	0	0

- **Attendance**: 10
- **Author's Position**: Favor
- **Outcome**: Consensus. Stronger than Poll 1.

Poll 3: We want a view with a convenient spelling for the semantics of `std::exclusive_scan` (which would always require an initial value).

Strongly Favor	Favor	Neutral	Against	Strongly Against
0	3	6	1	0

- **Attendance**: 10
- **Author's Position**: Against
- **Outcome**: Weak consensus.

Poll 4: We want a view with a convenient spelling for the semantics of `views::prescan` as proposed in the paper ([P3351R1]) (`std::exclusive_scan` plus the final sum).

Strongly Favor	Favor	Neutral	Against	Strongly Against
1	2	4	2	0

- **Attendance**: 10

- **Author's Position**: Favor
- **Outcome**: No consensus either way. Author's choice.

§ 8. Historical Records

> NOTE: These sections are now obsolete and no longer affects the current design. They only serve as a historical record.

§ 8.1. Why Three Adaptors? (Obsolete)

An immediately obvious question is why choose three names, instead of opting for a single `views::scan` adaptor with overloads that take initial seed and/or function parameter? Such a design will look like this (greatly simplified):

```
struct scan_closure
{
    template<ranges::input_range Rng, typename T, std::copy_constructible F
    requires /* ... */
    constexpr operator()(Rng&& rng, const T& init, Func func = {})
    { /* ... */ }

    template<ranges::input_range Rng, std::copy_constructible Fun = std::p
    requires /* ... */
    constexpr operator()(Rng&& rng, Func func = {})
    { /* ... */ }
};
inline constexpr scan_closure scan{};
```

Unfortunately, this does not work due to the same kind of ambiguity that caused `views::join_with` ([P2441R2]) to choose a different name instead of overload with `views::join`. Specifically, imagine someone writes a custom `vector` that replicates all the original interface, but introduced `operator+` to mean range concatenation and broadcast:

```
template<typename T>
struct my_vector : public std::vector<T>
{
```

```
    using std::vector<T>::vector;

    // broadcast: [1, 2, 3] + 10 = [11, 12, 13]
    friend my_vector operator+(my_vector vec, const T& value)
    {
        for (auto& elem : vec) elem += value;
        return vec;
    }
    friend my_vector operator+(const T& value, my_vector vec) { /* Same */

    // range concatenation: [1, 2, 3] + [4, 5] = [1, 2, 3, 4, 5]
    friend my_vector operator+(my_vector vec, const my_vector& vec2)
    {
        vec.append_range(vec2);
        return vec;
    }
    // operator+= implementation omitted
};
```

Although one could argue that this is a misuse of `operator+` overloading, this is definitely plausible code one could write. Now consider:

```
my_vector<int> vec{1, 2, 3}, vec2{4, 5};
views::partial_sum(vec); // [1, 3, 6]
vec2 | views::partial_sum(vec);
// [[1, 2, 3], [5, 6, 7], [10, 11, 12]] (!!)
```

The second invocation, `vec2 | views::partial_sum(vec)`, is equivalent to `partial_sum(vec2, vec)`, therefore interpreted as "using `vec` as the initial seed, and add each element of `vec2` to it". Unfortunately, we cannot differentiate the two cases, since they both invoke `partial_sum(vec)`. This ambiguity equally affects `views::scan`, since `scan(vec, plus{})` and `vec2 | scan(vec, plus{})` are also ambiguous.

There are several approaches we can adopt to handle this ambiguity:

Option 1: Bail out. Simply don't support the initial seed case, or just declare that anything satisfy `range` that comes in the first argument will be treated as the input range.

- Pros: No need to decide on new names.
- Cons: Losing a valuable use case that was accustomed by users since `std::exclusive_scan` exists. If the "declare" option is chosen, then potentially lose more use case like `my_vector` and

cause some confusion.

Option 2: Reorder arguments and bail out. We can switch the function and the initial seed argument, such that the signature is `scan(rng, func, init)`, and declare that anything satisfy `range` that comes in the first argument will be treated as the input range.

- Pros: No need to decide on new names.
- Cons:
  - Still have potential of conflict if a class that is both a range and a binary functor is passed in
  - Cannot support `partial_sum` with initial seed (discard or need new name)
  - Inconsistent argument order with `ranges::fold`

Option 3: Choose separate name for `scan` with and without initial seed.

- Pros: No potential of conflicts
- Cons: Need to decide on 1-2 new names and more wording effort

The author prefers Option 3 as it is the least surprising option that has no potential of conflicts. For now, the author decides that `scan` with an initial seed should be called `views::prescan`, as suggested in [P2760R1], and `partial_sum` should not support initial seed at all. The rationale for this decision is that people who want `partial_sum` with initial seed can simply call `prescan(init, std::plus{})`, so instead of coming up a name that is potentially longer and harder to memorize the author felt that this is the best approach.

8.1.1. SG9 Review, Wrocław (2024-11)

Members of SG9 expressed preference for Option 2, because of several reasons:

- `std::inclusive_scan` put function before init, which also enables init value to be an optional parameter. This change makes `views::scan` completely equivalent in principle to `std::inclusive_scan` in functionality (sans the associativity issue).
- The shown conflict case is too arcane and unlikely to cause actual problems in practice.

§ References

§ Normative References

[N5001]

Thomas Köppe. *Working Draft, Programming Languages — C++*. 17 December 2024. URL: https://wg21.link/n5001

[P2760R1]

Barry Revzin. *A Plan for C++26 Ranges*. 15 December 2023. URL: https://wg21.link/p2760r1

[P3351R1]

Yihe Li. *views::scan*. 9 October 2024. URL: https://wg21.link/p3351r1

§ Informative References

[N4988]

Thomas Köppe. *Working Draft, Programming Languages — C++*. 5 August 2024. URL: https://wg21.link/n4988

[P1642R11]

Ben Craig. *Freestanding Library: Easy [utilities], [ranges], and [iterators]*. 1 July 2022. URL: https://wg21.link/p1642r11

[P1729R5]

Elias Kosunen, Victor Zverovich. *Text Parsing*. 15 October 2024. URL: https://wg21.link/p1729r5

[P2214R2]

Barry Revzin, Conor Hoekstra, Tim Song. *A Plan for C++23 Ranges*. 18 February 2022. URL: https://wg21.link/p2214r2

[P2322R6]

Barry Revzin. *ranges::fold*. 22 April 2022. URL: https://wg21.link/p2322r6

[P2441R2]

Barry Revzin. *views::join_with*. 28 January 2022. URL: https://wg21.link/p2441r2

[P2846R5]

Corentin Jabot. *reserve_hint: Eagerly reserving memory for not-quite-sized lazy ranges*. 27 November 2024. URL: https://wg21.link/p2846r5

[P3117R1]

Zach Laine, Barry Revzin, Jonathan Müller. *Extending Conditionally Borrowed*. 15 December 2024. URL: https://wg21.link/p3117r1

[P3136R1]

Tim Song. *Retiring niebloids*. 18 November 2024. URL: https://wg21.link/p3136r1

[P3179R4]

Ruslan Arutyunyan, Alexey Kukanov, Bryce Adelstein Lelbach. *C++ parallel range algorithms*. 11 December 2024. URL: https://wg21.link/p3179r4